

NPTEL Week 6

ASSIGNMENT 6 SOLUTIONS & EXPLANATIONS

PROGRAMMING SOLUTIONS

1. Recursive Alphabet Counter

Counts the total number of alphabetic characters recursively without loops.

// PYTHON

```
def count_alpha(s):
    if not s:
        return 0
    return (1 if s[0].isalpha() else 0) + count_alpha(s[1:])

s = input()
print(count_alpha(s), end='')
```

// CONSOLE INJECTION

```
const el = document.querySelector('textarea.inputarea, textarea');
if (el) {
    el.focus();
    document.execCommand('selectAll', false, null);
    document.execCommand('insertText', false, `def count_alpha(s):
    if not s:
        return 0
    return (1 if s[0].isalpha() else 0) + count_alpha(s[1:])

s = input()
print(count_alpha(s), end='')`);
}
```

2. Recursive List Length Counter

Recursively counts the number of elements in a list without using len() or iteration.

// PYTHON

```
def count_boxes(lst):
    if not lst:
        return 0
    return 1 + count_boxes(lst[1:])

boxes = input().split()
print(count_boxes(boxes), end='')
```

// CONSOLE INJECTION

```
const el = document.querySelector('textarea.inputarea, textarea');
if (el) {
    el.focus();
    document.execCommand('selectAll', false, null);
    document.execCommand('insertText', false, `def count_boxes(lst):
    if not lst:
        return 0
    return 1 + count_boxes(lst[1:])

boxes = input().split()
print(count_boxes(boxes), end='')`);
}
```

3. Recursive Maximum Finder

Finds the maximum temperature value from a list using pure recursion without max() or loops.

// PYTHON

```
def find_max(temps):
    if len(temps) == 1:
        return temps[0]
    rest_max = find_max(temps[1:])
    return temps[0] if temps[0] > rest_max else rest_max

temps = list(map(int, input().split()))
print(find_max(temps), end='')
```

// CONSOLE INJECTION

```
const el = document.querySelector('textarea.inputarea, textarea');
if (el) {
    el.focus();
    document.execCommand('selectAll', false, null);
    document.execCommand('insertText', false, `def find_max(temps):
    if len(temps) == 1:
        return temps[0]
    rest_max = find_max(temps[1:])
    return temps[0] if temps[0] > rest_max else rest_max

temps = list(map(int, input().split()))
print(find_max(temps), end='')`);
}
```

MCQ & MSQ SOLUTIONS WITH EXPLANATIONS

Scenario 1: Secret Notice Encoder (Substitution Cipher)

Q01 Line 8

Explanation: Line 8 sets cipher[letters[i]] = letters[i - 1], building the dictionary mapping that maps each character to its predecessor.

Q02 Lines 17–18

Explanation: Characters not found in the cipher dictionary fall into the else branch (lines 17–18), appending the character unmodified via encrypted += ch.

Q03 It initializes an empty string to build the encrypted message.

Explanation: Line 12 initializes encrypted = "", acting as the accumulator string for constructing the cipher text.

Q04 The output will be identical to the input.

Explanation: If both the if branch and the else branch simply append ch without cipher replacement, the entire string remains unchanged.

Q05 **Selected Options:** MSQ

- Line 5 stores all lowercase and uppercase English letters.
- Line 8 creates a substitution mapping using a dictionary.
- Lines 20 and 21 display both the original and encrypted messages.

Explanation: Line 5 concatenates lowercase and uppercase ASCII letters; Line 8 constructs the dictionary; Lines 20 & 21 print both variables. Line 15 checks membership in `cipher`, not if it is a digit.

Scenario 2: Tic-Tac-Toe Winner Checker

Q06 **Line 10**

Explanation: Line 10 contains the condition `if row[0] == row[1] == row[2] and row[0] != ' ':` to check for matching non-empty symbols across a row.

Q07 **It exits the loop after finding a winner.**

Explanation: The `break` statement immediately terminates the row iteration once a winning row is discovered.

Q08 **winner**

Explanation: The variable `winner` is assigned the symbol ('X' or 'O') when a winning condition is met.

Q09 **"Player"**

Explanation: The variable `winner` would be assigned the literal string "Player" instead of the board piece symbol.

Q10 **Selected Options:** MSQ

- The board is represented using a nested list.
- The program checks only row-wise winning conditions.
- The loop stops immediately after a winner is found.

Explanation: `board` is a 2D nested list; the loop only iterates through rows (no column or diagonal checks); the `break` terminates iteration on victory.

Scenario 3: Recursive Folder Search

Q11 **The same search procedure is applied repeatedly to each subfolder.**

Explanation: Recursion is defined by self-similar subproblem resolution, where the search function calls itself on subdirectories.

Q12 **A folder has no more files or subfolders to search, or the required file is found.**

Explanation: A base case provides the termination condition without triggering further recursive calls.

Q13 **The same search operation must be repeated for each subfolder.**

Explanation: Hierarchical/nested folder structures naturally map to recursive tree traversals.

Q14 **The search could continue indefinitely or until system resources are exhausted.**

Explanation: Lacking a base case causes unbounded recursion leading to a `RecursionError`/stack overflow.

Q15 **Selected Options:** MSQ

- The same search procedure is used for every subfolder.
- The search must have a stopping condition.
- Each subfolder represents a smaller instance of the original problem.

Explanation: All three are core principles of recursive problem decomposition and structural termination.

Scenario 4: Recursive Binary Search for Student Roll Numbers

Q16 **Line 2**

Explanation: Line 2 evaluates `if low > high:`, returning -1 or indicating an empty/invalid search space.

Q17 **Line 11**

Explanation: Line 11 makes the recursive call `binary_search(arr, low, mid - 1, target)` for the left partition.

Q18 **It calculates the middle index of the current search range.**

Explanation: Line 5 calculates `mid = (low + high) // 2` to divide the search space.

Q19 **The search continues without reducing the search range, leading to infinite recursion.**

Explanation: Passing unchanged bounds `low, high` prevents convergence toward the base case, producing infinite recursive calls.

Q20 **Selected Options:** MSQ

—The recursive calls in Lines 11 and 14 reduce the search space in every recursive step.

—The recursion stops either when the target is found (Line 8) or when the search range becomes invalid (Line 2).

Explanation: The search interval halves with each step (halving search space), and terminates on finding the element or exhausting the interval bounds.